

Library Management Spring Boot Project

Entity Relationship Design

We have two main entities: **Author** and **Book**. - **Author**: Contains an `id`, `name`, and a list of books. - **Book**: Contains an `id`, `title`, `genre`, and an `author`.

The relationship between the two is **OneToMany** from **Author** to **Book** and **ManyToOne** from **Book** to **Author**. This means one author can write many books, while each book corresponds to exactly one author.

Implementation Details

Populate Database

We implemented a `DataInitializer` that utilizes Spring's `CommandLineRunner` interface. Upon the application startup, it populates the database with 10 sample **Author** elements and 10 sample **Book** elements if the author table is empty.

Create Operation

- Created an `add.jsp` view that includes an HTML form to add a new **Book**.
- The controller mapping `@PostMapping("/add")` receives the required parameters, checks for constraints, and delegates book creation to the `LibraryService`.
- Custom error handling (`DataIntegrityViolationException`) handles the case if an invalid `authorId` or missing required constraint occurs, redirecting the user appropriately.

Read Operation

- We have the `list.jsp` to display all existing books securely using JSTL.
- To display book details alongside author names, we added a custom query inside the `BookRepository` that runs an `INNER JOIN`:

```
java @Query("SELECT new com.example.library.dto.BookAuthorDTO(b.id, b.title, b.genre, a.name) FROM Book b INNER JOIN b.author a") List<BookAuthorDTO> findBooksWithAuthors();
```
- This data is directly passed onto JSP views through the model in `LibraryController`.

Update Operation

- Included an edit link for every row in the `list.jsp` connecting to `/update/{id}`.
- Handled safely via a GET mapping that populates the `update.jsp` with current values. Wait, upon submission, a POST mapping modifies the same record using `findById`.

Challenges Faced

- **View Configuration in Spring Boot 3.2.x**: JSP views require precise integration with embedded tomcat (`tomcat-embed-jasper`) and Jakarta EE specs instead of the old javax dependencies. Correcting the `.pom` with `jakarta.servlet.jsp.jstl` resolved the compatibility errors.
- **Join Operation**: Representing an inner join query purely as objects needed the `BookAuthorDTO` so it can be extracted and sent to the view smoothly.

GitHub URL

(The user must upload this code to a new Github repository and place the link here)

How to Generate PDF: You can use your browser to view this markdown (via any Markdown previewer) and choose 'Print -> Save as PDF'.